

14장 포인터와 함수

1. call by value vs. call by reference
2. 함수의 인자로 배열 전달하기
3. 포인터 대상의 const 선언



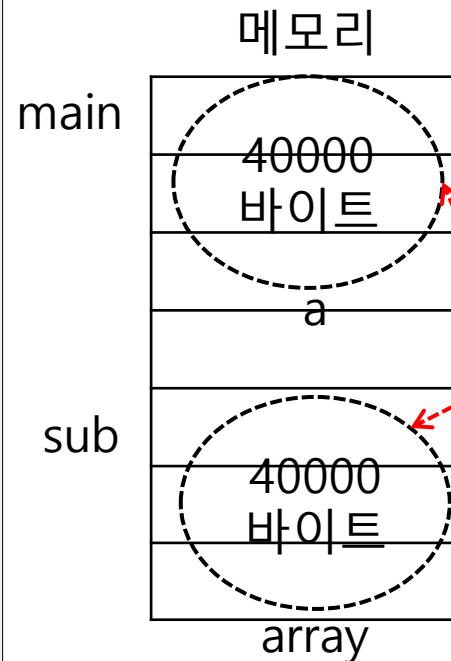
함수의 인자에 배열을 전달하는 경우

3

- 값에 의한 호출 사용시 메모리를 많이 사용하고 인자를 매개변수에 복사하는데 시간이 많이 걸림 -> 성능저하
- 함수의 인자가 배열인 경우는 주소에 의한 호출만 허용됨

```
int main()
{
    int a[10000];
    ...
    sub(배열값전달);
    ...
}

void sub(int array[10000])
{
    ...
}
```



1. 똑같은 대용량 데이터가 2군데 존재
2. 대용량 매개변수를 초기화하는데 많은 시간 필요



함수의 인자에 배열을 전달하는 경우

4

- 함수의 인자가 배열인 경우 함수의 매개변수를 포인터 변수로 선언
- 함수 호출시 인자에 배열의 시작 주소(배열명)를 전달
- 배열의 시작 주소만 알면 배열의 모든 원소를 접근가능

```
int main()
{
    int a[10000];
    ...
    f(a); //배열명을 인자로 전달
    ...
}

void f(int* pt) //주소에 의한 호출
{
    ...
}
```

```
int main()
{
    int a[10000];
    ...
    f(a);
    ...
}

void f(int array[ ])
{
    ...
}
```

int array*
*배열처럼 선언해도 되지만
컴파일러에 의해 포인터로
자동변환 됨*



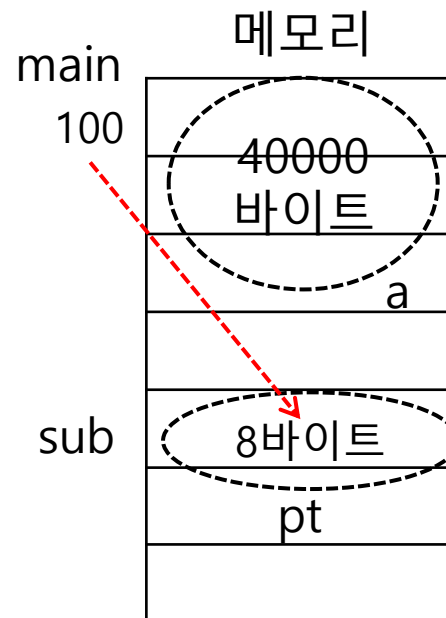
함수의 인자에 배열을 전달하는 경우

5

- 매개변수가 포인터이므로 8바이트 공간만 할당 -> 메모리절약
- 배열의 시작주소로 매개변수 초기화 -> 속도개선

```
int main()
{
    int a[10000];
    ...
    f(a); //배열명을 인자로 전달
    ...
}

void f(int* pt) //주소에 의한 호출
{
    ...
}
```





예제: 배열 출력하기 -> 포인터표현사용

6

```
#include <stdio.h>
void ShowArray(int* param, int len);
int main(void)
{
    int arr1[3] = { 1, 2, 3 };
    int arr2[5] = { 4, 5, 6, 7, 8 };
    ShowArray(arr1, 3);
    ShowArray(arr2, 5);
    return 0;
}
void ShowArray (int* param, int len)
{
    int i;
    for (i = 0; i < len; i++) printf("%d ", *(param+i));
    printf("\n");
}
```

1	2	3		
4	5	6	7	8



예제: 배열 출력하기 -> 배열표현 사용

7

```
#include <stdio.h>
void ShowArray(int param[ ], int len);
int main(void)
{
    int arr1[3] = { 1, 2, 3 };
    int arr2[5] = { 4, 5, 6, 7, 8 };
    ShowArray(arr1, 3);
    ShowArray(arr2, 5);
    return 0;
}
void ShowArray (int param[ ], int len)
{
    int i;
    for (i = 0; i < len; i++) printf("%d ", param[i]);
    printf("\n");
}
```

```
1 2 3
4 5 6 7 8
```



예제: 평균값 구하기

8

```
#include <stdio.h>
void print_array(int* pt, int n);
double get_avg(int* pt, int n);
int main(void)
{
    int i, data[5] = { 10, 20, 30, 40, 50 }, result;
    int len = sizeof(data)/sizeof(int);
    print_array(data, len);
    result = get_avg(data, len);
    printf("평균 = %lf\n", result);
    return 0;
}
```

10 20 30 40 50
평균 = 30.000000



예제: 평균 구하기

9

```
void print_array(int* pt, int n)
{
    -> int pt[ ]
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", pt[i]);    // pt[i] -> *(pt + i);
}

double get_avg(int* pt, int n)
{
    -> int pt[ ]
    int i;
    double sum = 0.0;
    for (i = 0; i < n; i++) sum += pt[i];    // sum += *(pt + i);
    return sum / n;
}
```

10 20 30 40 50
평균 = 30.000000



예제: 배열요소의 값 변경하기

10

```
#include <stdio.h>
void ShowArray(int* param, int len);
void AddArray(int* param, int len, int add);
int main(void)
{
    int arr[3] = { 1, 2, 3 };
    AddArray(arr, sizeof(arr) / sizeof(int), 1);
    ShowArray(arr, sizeof(arr) / sizeof(int));

    AddArray(arr, sizeof(arr) / sizeof(int), 2);
    ShowArray(arr, sizeof(arr) / sizeof(int));

    AddArray(arr, sizeof(arr) / sizeof(int), 3);
    ShowArray(arr, sizeof(arr) / sizeof(int));
    return 0;
}
```

2	3	4
4	5	6
7	8	9



예제: 배열요소의 값 변경하기

11

```
void ShowArray(int* param, int len)
{
    int i;
    for (i = 0; i < len; i++) printf("%d ", param[i]);
    printf("\n");
}

void AddArray(int* param, int len, int add)
{
    int i;
    for (i = 0; i < len; i++) param[i] += add;
}
```

2	3	4
4	5	6
7	8	9

주소를 받기 때문에 다른
함수에 선언된 배열의 값을
변경하는 것도 가능함



예제: 최소값 구하기

12

```
#include <stdio.h>
int get_min(int* array, int n);
int main(void)
{
    int grade[5];
    int i, min;
    for(i = 0; i < 5; i++)
    {
        printf("성적을 입력하시오: ");
        scanf("%d", &grade[i]);
    }
    min = get_min(grade, 5);
    printf("최소값은 %d입니다.\n", min);
    return 0;
}
```

성적을 입력하시오: 50 <엔터>
성적을 입력하시오: 10 <엔터>
성적을 입력하시오: 30 <엔터>
성적을 입력하시오: 20 <엔터>
성적을 입력하시오: 60 <엔터>
최소값은 10입니다.



예제: 최소값 구하기

13

```
int get_min(int* array, int n)           // int get_min(int array[ ], int n)
{
    int i, min;
    min = *array;                         // *array => array[0];
    for(i = 1; i < n; i++)
        if(*(array+i) < min ) min = *(array+i); // *(array+i) => array[i];

    return min;
}
```



함수의 리턴값이 2개 이상인 경우

14

- C언어 함수는 반환값을 1개만 정의 가능
- 반환값이 2개 이상인 경우 -> 매개변수를 이용하여 반환
- 반환값을 저장할 변수(다른 함수의 지역변수)의 주소를 인자로 받아서 처리(주소에 의한 호출)

```
int add(int a, int b)
{
    int result;
    result = a+b;
    return result;
}
```

```
void add(int a, int b, int* res)
{
    *res = a+b;
}
```



예제: 리터갈을 매개변수로 처리

15

```
#include <stdio.h>
int add(int a, int b);
int main(void)
{
    int result;
    result = add(2,3);
    printf("합 = %d\n", result);
    return 0;
}
int add(int a, int b)
{
    int result;
    result = a+b;
    return result;
}
```

```
#include <stdio.h>
void add(int a, int b, int* res);
int main(void)
{
    int result;
    add(2,3, &result);
    printf("합 = %d\n", result);
    return 0;
}
void add(int a, int b, int* res)
{
    *res = a+b;
}
```

다른 함수의 지역변수의 값을
변경하므로 주소에 의한
호출사용



예제: 반환값이 2개인 함수

16

```
#include <stdio.h>
void get_maxmin(int* array, int* max, int* min);
int main(void)
{
    int grade[5]={50,10,30,20,60};
    int max, min;
    get_maxmin(grade, &max, &min);
    printf("최대값은 %d, 최소값은 %d\n", max, min);
    return 0;
}
```

최대값은 60, 최소값은 10



예제: 반환값이 2개인 함수

17

```
void get_maxmin(int* array, int* max, int* min)
{
    int i, big, small;
    small = *array;
    for(i = 1; i < 5; i++)
        if(*(array+i) < small) small = *(array+i);
    big = *array;
    for(i = 1; i < 5; i++)
        if(*(array+i) > big) big = *(array+i);
    *max = big;
    *min = small;
}
```

최대값은 60, 최소값은 10

- 함수의 매개변수로 포인터를 사용해야 하는 경우(주소에 의한 호출)
 - ▣ 다른 함수에서 선언된 지역 변수의 값을 변경하고 싶은 경우
 - ▣ 배열을 함수의 인자에 전달하는 경우
 - ▣ 함수의 리턴 값이 2개 이상인 경우 -> 첫번째 경우와 같음



- 변수 선언 시 앞에 const 붙이면 변수를 상수처럼 만듦 -> 변수의 값을 바꿀 수 없음(교재 5장 124페이지)
- const 사용 시 반드시 선언과 동시 초기화 할 것
- 변수처럼 사용가능, 변수에 적용가능한 모든 연산 가능(&, 형변환)

```
const double tax_rate = 0.12;  
tax_rate = 0.50;  
const int a;
```

```
// 변수값이 0.12로 고정됨  
// 컴파일 에러  
// 컴파일 에러, 초기값 없음
```



상수 표현 방법

20

□ 상수를 표현하는 3가지 방법

// 리터럴 상수 : 숫자나 문자값 그대로 쓰는 것

```
x = 0.12;
```

// 기호상수 : 리터럴 상수를 기호로 정의한 것(26장에서 배움)

```
#define TAX_RATE 0.12
```

```
x = TAX_RATE;
```

// const 변수 : 리터럴 상수를 const 변수로 정의한 것

```
const double TAX_RATE = 0.12;
```

```
x = TAX_RATE ;
```



예제: const 변수

21

```
#include <stdio.h>
int main(void)
{
    int income=0;
    double tax;
    const double TAX_RATE = 0.12;

    income = 456;
    tax = income * TAX_RATE;
    printf("세금은 : %.1lf입니다.\n", tax);
    return 0;
}
```

세금은 : 54.7입니다.

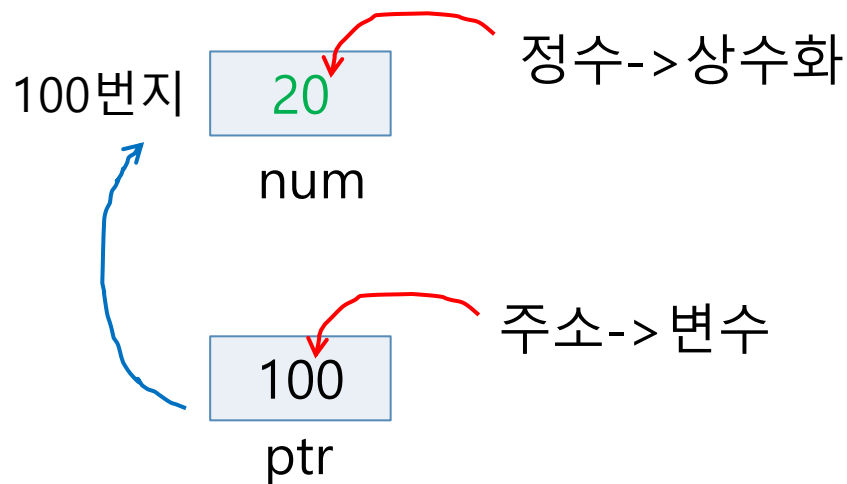


const를 사용한 포인터

22

```
int num=20;  
const int* ptr = &num;  
*ptr = 30;           //컴파일 에러  
num = 40;            //정상
```

- 포인터가 가리키는 변수의 값을 바꿀 수 없음(상수화) -> 함수의 매개변수에 많이 사용됨(21장)
- 포인터 변수의 값은 변경가능



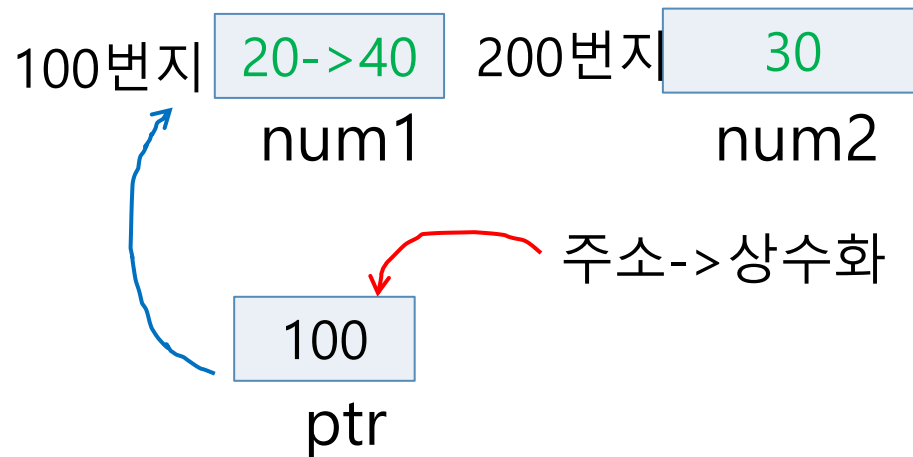


const를 사용한 포인터

23

```
int num1=20, num2=30;  
int* const ptr = &num1;  
ptr = &num2;           //컴파일 에러  
*ptr = 40;             //정상
```

- 포인터 변수값을 바꿀 수 없음(상수화)
- 포인터가 가리키는 변수의 값은 변경가능



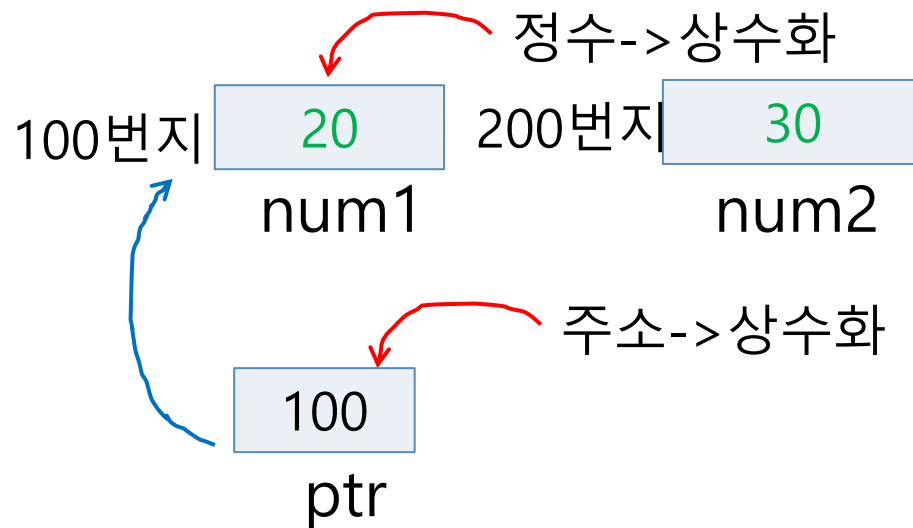


const를 사용한 포인터

24

- 포인터가 가리키는 변수와 포인터 변수값을 모두 상수화

```
int num1=20,num2=30;  
const int* const ptr = &num1;  
ptr = &num2;           //컴파일 에러  
*ptr = 40;             //컴파일 에러
```





const 사용한 변수

25

```
#include <stdio.h>
int main(void)
{
    int a = 10, b = 20;
    const int* pa = &a;

    printf("변수a값:%d\n", *pa);
    pa = &b;
    printf("변수b값:%d\n", *pa);
    a = 20;
    /*pa=20;
    printf("변수a값:%d\n", *pa);
    return 0;
}
```

변수a값:10
변수b값:20
변수a값:20



실습과제 1

26

- 주소에 의한 호출을 사용해야하는 3가지 경우를 설명하라
- 최대값 구하는 알고리즘을 설명하라
- `const` 선언을 사용하는 이유를 설명 하시오. 교재 322~323페이지 참고할 것



실습과제2

27

- 최소값 구하기 예제를 참고하여 아래 결과가 나오도록 코드를 수정 하시오. 최대값을 구하는 부분은 반드시 함수로 작성하시오.

정수5개를 입력 하시오.

50<엔터>

20<엔터>

30<엔터>

40<엔터>

10<엔터>

최대값은 50입니다.



실습과제3

28

- 키보드로 부터 5개의 정수를 입력 받아 배열(data)에 저장해주는 부분을 함수(get_data)로 작성하시오. 아래 코드에서 선언, 호출, 정의 부분을 실행 결과를 고려하여 완성하시오.

```
#include <stdio.h>
```

```
//get_data선언
```

```
int main(void)
```

```
{
```

```
    int i, data[5];
```

```
    //get_data 호출
```

```
    for(i = 0; i < 5; i++)
```

```
        printf("%d번째 data:%d\\n", i+1, data[i]);
```

```
    return 0;
```

```
}
```

```
// get_data 정의
```

1번째 data를 입력하시오: 50 <엔터>

2번째 data를 입력하시오: 10 <엔터>

...

5번째 data를 입력하시오: 60 <엔터>

1번째 data: 50

2번째 data: 10

...

5번째 data: 60

- 키보드로 부터 실수를 입력 받아 그것의 정수부와 소수부를 구하여 출력해주는 프로그램을 작성하시오.
- 정수부와 소수부를 구하여 리턴하는 부분을 1개의 함수로 작성할 것
- 화면에 입출력하는 부분은 메인함수에서 작성할 것
- 함수의 정의, 호출, 선언을 모두 작성하시오.

실수를 입력하시오 : 3.14159<엔터>
정수부 : 3
소수부 : 0.14159



- 교재 324페이지 문제1번 또는 문제2번 중에 1문제를 푸세요.



- 15장의 도전 1~5번 중 3문제를 푸시오.



과제제출방법

32

- 아래 주소의 작성방법대로 github에 제출할 것
 - ▣ <https://github.com/2sungryul/cprogramming/tree/main>
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```